

SIMATS ENGINEERING — ASSESSMENT TOOL 3 SOLUTIONS

COURSE: CSA16 - Data Warehousing and Data Mining | CO4: Clustering Algorithms in Real-World Scenarios

Q1. Dataset: Customer Data

Bloom's Level: BL3 — 10 Marks

Question: Compare K-means and hierarchical clustering using this dataset and explain differences in performance and scalability.

1. DATASET OVERVIEW & STRUCTURAL MAPPING

The dataset consists of 5 customer records (C1 to C5) across two numerical dimensions: *Income* (₹) and *Spending Score*:

- **C1:** (30000, 60) | **C2:** (60000, 40) | **C3:** (50000, 70) | **C4:** (28000, 65) | **C5:** (65000, 35)

Natural groupings emerge based on distance proximity: **Cluster A (Low Income, High Spending):** C1 and C4; **Cluster B (High Income, Moderate/Low Spending):** C2, C3, and C5.

2. ALGORITHMIC EXECUTION & DETAILED COMPARISON

- **K-Means Clustering:** Partitions data into K distinct clusters (e.g., K=2) by minimizing the Sum of Squared Errors (SSE). Income values reach ₹65,000 while Spending Scores range between 35 and 70. Because K-means relies heavily on Euclidean distance, the larger magnitude of Income will dominate distance calculations unless features are explicitly normalized using Min-Max scaling. Converges in linear time $O(K \cdot N \cdot d)$.
- **Hierarchical Clustering (Agglomerative):** Constructs a bottom-up binary tree (dendrogram). It begins with each point in its own cluster and iteratively merges the closest pair of clusters based on a proximity metric. Does not require pre-specifying K. Cutting the dendrogram at a high distance threshold yields 2 clusters ({C1, C4}, {C2, C3, C5}), revealing sub-hierarchies such as C2 and C5 being exceptionally close.

3. PERFORMANCE & SCALABILITY EVALUATION

Performance Metric	K-Means Clustering	Hierarchical Clustering
Time Complexity	$O(N \cdot K \cdot I \cdot d)$ — Linearly scalable with number of records N.	$O(N^3)$ standard or $O(N^2 \log N)$ optimized — Poor scalability for large N.
Space Complexity	$O(N \cdot d + K \cdot d)$ — Memory efficient; stores only dataset and centroids.	$O(N^2)$ — High memory cost due to maintaining pairwise proximity matrix.
Cluster Shape Assumptions	Assumes spherical, convex clusters of similar size.	Can capture arbitrarily shaped clusters depending on linkage criteria.
Determining Clusters	Requires K to be known or chosen a priori (e.g., via Elbow Method).	Produces a full tree hierarchy; K selected after inspecting dendrogram.
Scalability to Large Datasets	Highly scalable to millions of customer records.	Unsuitable for large datasets; useful for detailed exploration of small data.

Question: Compare K-means and EM algorithms in handling overlapping clusters for the given dataset.

1. DATASET REPRESENTATION & CLUSTER OVERLAP ANALYSIS

The dataset presents vector space representations (normalized term frequency / topic distribution) for 5 documents across 3 words:

- **D1:** (0.2, 0.5, 0.3) | **D2:** (0.1, 0.6, 0.3) | **D3:** (0.8, 0.1, 0.1) | **D4:** (0.75, 0.15, 0.1) | **D5:** (0.2, 0.4, 0.4)

D3 and D4 strongly belong to Topic 1 (dominant Word1). D1, D2, and D5 represent a secondary topic group with mixed representations of Word2 and Word3. D1 and D5 sit on boundary regions between pure topics, showing soft degrees of membership across features.

2. COMPARATIVE ALGORITHMIC EXECUTION

- **K-Means Clustering (Hard Assignment):** K-means forces a strict partitioning constraint: each document belongs exclusively to a single cluster indicator $w_{ik} \in \{0, 1\}$. For transitional boundary documents like D1 and D5, K-means makes an arbitrary hard boundary cut, failing to capture that D1 shares features heavily with D2 while also sharing equal Word3 distribution with D5.
- **Expectation-Maximization / Gaussian Mixture Models (Soft Assignment):** EM assumes data is generated from a mixture of underlying probability distributions. Instead of crisp labels, EM computes a posterior probability distribution $P(\text{Cluster } k \mid D_i)$ during the E-step and updates mean vectors and covariance matrices during the M-step. For D1 (0.2, 0.5, 0.3), EM assigns explicit probabilistic membership across components (e.g., $P(\text{Cluster } 1 \mid D1) = 0.15$ and $P(\text{Cluster } 2 \mid D1) = 0.85$).

3. ALGORITHMIC COMPARISON

Feature / Aspect	K-Means Algorithm	Expectation-Maximization (EM)
Membership Type	Hard clustering (Binary: 0 or 1).	Soft probabilistic clustering ([0.0, 1.0] probability range).
Handling Overlap	Forces boundary points into a single cluster, losing ambiguity.	Naturally models overlapping cluster boundaries via mixture probabilities.
Mathematical Basis	Minimum distance optimization (Centroids via Mean).	Maximum Likelihood Estimation (MLE) via probability distributions.
Cluster Geometry	Standard Euclidean distance yields rigid, hyper-spherical clusters.	Covariance parameters allow elliptical and elongated clusters.
Suitability for Text	Poor fit when documents cover multiple topics simultaneously.	Highly effective for topic modelling where documents span themes.

Question: Compare Euclidean and Manhattan distance metrics and analyze their impact on clustering results.

1. DATASET OVERVIEW & MATHEMATICAL DEFINITIONS

The dataset contains spatial geographical coordinates (Latitude, Longitude) for 5 locations: **P1:** (13.08, 80.27), **P2:** (13.10, 80.25), **P3:** (12.97, 80.22), **P4:** (13.05, 80.30), **P5:** (13.00, 80.20).

Euclidean Distance (L2 Norm): $d(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$

Manhattan Distance (L1 Norm / City-Block): $d(A, B) = |x_2 - x_1| + |y_2 - y_1|$

2. NUMERICAL CALCULATIONS

- **Distance P1(13.08, 80.27) to P2(13.10, 80.25):**
 $\Delta\text{Lat} = |13.10 - 13.08| = 0.02$ | $\Delta\text{Lon} = |80.25 - 80.27| = 0.02$
Euclidean Distance: $\sqrt{(0.02^2 + 0.02^2)} = \sqrt{0.0008} \approx \mathbf{0.02828}$
Manhattan Distance: $0.02 + 0.02 = \mathbf{0.04000}$
- **Distance P1(13.08, 80.27) to P3(12.97, 80.22):**
 $\Delta\text{Lat} = |12.97 - 13.08| = 0.11$ | $\Delta\text{Lon} = |80.22 - 80.27| = 0.05$
Euclidean Distance: $\sqrt{(0.11^2 + 0.05^2)} = \sqrt{0.0121 + 0.0025} \approx \mathbf{0.12083}$
Manhattan Distance: $0.11 + 0.05 = \mathbf{0.16000}$

3. IMPACT ON CLUSTERING RESULTS

- **Sensitivity to Axis Traversal:** Euclidean distance calculates direct spatial proximity ("as the crow flies"). Manhattan distance measures distance along orthogonal grid pathways, better modeling real-world city road networks.
- **Effect of Large Coordinate Differences:** Euclidean distance squares spatial differences $(\Delta x)^2 + (\Delta y)^2$, causing large single-axis variances to exert a disproportionately strong influence on distance, pulling centroids toward outliers. Manhattan distance weights individual component differences linearly, making it more robust against isolated extreme coordinate shifts.
- **Cluster Geometry:** Euclidean distance in algorithms like K-means forms spherical/circular spatial clusters, whereas Manhattan distance produces diamond- or hyper-octahedral-shaped cluster boundaries.

Question: Compare agglomerative and divisive clustering approaches and explain their suitability for this dataset.

1. DATASET OVERVIEW & SEGMENT IDENTIFICATION

Data consists of 5 retail products characterized by two numerical attributes: *Sales* (₹) and *Frequency*: **P1:** (50000, 20), **P2:** (30000, 10), **P3:** (52000, 22), **P4:** (25000, 8), **P5:** (32000, 12).
Identified groups: **High-Performing Segment:** P1 and P3; **Moderate/Low Segment:** P2, P4, and P5.

2. METHODOLOGICAL COMPARISON

- **Agglomerative Hierarchical Clustering (Bottom-Up):** Begins by treating each of the 5 products as an individual cluster. *Step 1:* Computes distance matrix, merging closest pair P1 and P3 (Sales diff = ₹2000, Freq diff = 2) into C_13 = {P1, P3}. *Step 2:* Merges next closest pair P2 and P5 into C_25 = {P2, P5}. *Step 3:* Merges P4 into C_25, forming C_254 = {P2, P5, P4}. *Step 4:* Combines C_13 and C_254 into a single unified root cluster.
- **Divisive Hierarchical Clustering (Top-Down):** Begins with a single root cluster containing all 5 items: {P1, P2, P3, P4, P5}. *Step 1:* Splits based on maximal variance (Sales feature variance spans ₹25,000 to ₹52,000). *Step 2:* Splits root cluster into two distinct sub-clusters: High-tier {P1, P3} and Low-tier {P2, P4, P5}. *Step 3:* Iteratively subdivides lower-tier groups further until all products sit in individual leaf nodes.

3. SUITABILITY ANALYSIS FOR RETAIL PRODUCT DATASET

- **Agglomerative Approach:** Highly suitable for bottom-up product grouping. It naturally pairs items with similar sales figures and purchase frequencies (P1 with P3, and P2 with P5). Because it builds fine-grained combinations incrementally, it works well for identifying localized sub-categories in retail catalog structures.
- **Divisive Approach:** Highly effective when the primary goal is high-level portfolio segmentation (e.g., quickly separating overall "High Value" products from "Low Value" products). However, divisive algorithms are computationally more complex ($O(2^N)$ possibilities per split without heuristics), making them less common for small catalogs unless a top-down global perspective is strictly required.

Question: Compare centroid-based and density-based clustering techniques and analyze their effectiveness on mixed data types.

1. DATASET BREAKDOWN

Contains 5 items evaluated across mixed attribute types: *Price* (₹) (Continuous numeric), *Category* (Nominal 'A'/'B'), *Rating* (Continuous scale 3.5 - 4.7):
• **I1:** (500, 'A', 4.5) | **I2:** (300, 'B', 3.8) | **I3:** (550, 'A', 4.7) | **I4:** (250, 'B', 3.5) | **I5:** (520, 'A', 4.6)

2. COMPARISON OF CORE TECHNIQUES

- **Centroid-Based Clustering (e.g., K-Means):** Defines clusters using a central mean vector. Computing an arithmetic mean requires continuous numeric dimensions. Calculating the mean of a nominal categorical variable like Category ('A' vs 'B') is mathematically undefined without encoding. Restricted to finding convex, spherical clusters.
- **Density-Based Clustering (e.g., DBSCAN):** Defines clusters as continuous regions of high density separated by regions of low density, using parameters ϵ (Eps radius) and MinPts. Relies entirely on a distance metric $d(x, y)$ to count points within radius ϵ . Can discover arbitrarily shaped clusters and explicitly filter out sparse noise.

3. EFFECTIVENESS ON MIXED DATA TYPES & ADAPTATION STRATEGIES

Feature	Centroid-Based (K-Means / K-Prototypes)	Density-Based (DBSCAN / HDBSCAN)
Handling Nominal Features	Fails natively on categorical fields like Category ('A', 'B'). Mean cannot be computed on strings.	Fails natively if standard Euclidean distance is used. Requires customized mixed-distance functions.
Adaptation Strategy	Requires K-Prototypes algorithm (combines K-Means for numeric and K-Modes for nominal features).	Requires distance metrics like Gower's Distance Matrix , which calculates distance per feature type.
Handling Scale Differences	High variance in Price (250 to 550) dominates Rating (3.5 to 4.7) unless standardized.	ϵ -radius parameters become uninterpretable if feature scales differ wildly without normalization.
Noise & Outlier Handling	Outliers distort calculated mean centroids, shifting cluster locations.	Robustly categorizes sparse, isolated points as noise rather than forcing them into a cluster.

4. OPTIMAL PREPROCESSING & PIPELINE

1. Convert *Category* using One-Hot Encoding or apply **Gower's Distance**.
2. Apply Min-Max Normalization to *Price* and *Rating* to scale both metrics smoothly to [0, 1].
3. Use **K-Prototypes** for centroid-based clustering, or compute pairwise **Gower's Distance Matrix** and feed into **DBSCAN** or **Hierarchical Clustering**.